محمد محمد ايمن مسعد مسعد

- Ram, PISO, SIPO, ALU

Ram

ram.v

```v
module ram #(
    parameter ADDR_WIDTH = 8,
    parameter DATA_WIDTH = 20
)(
    input  wire                  clk,
    input  wire                  rst_n,
    input  wire                  wr_en,
    input  wire [ADDR_WIDTH-1:0] addr,
    input  wire [DATA_WIDTH-1:0] din,
    input  wire                  rd_en,
    output reg  [DATA_WIDTH-1:0] dout,
    output reg                   valid
);

    reg [DATA_WIDTH-1:0] mem [0:(1<<ADDR_WIDTH)-1];

    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            dout  <= {DATA_WIDTH{1'b0}};
            valid <= 1'b0;
        end
        else begin
            valid <= 1'b0;

            if (wr_en)
                mem[addr] <= din;

            if (rd_en) begin
                dout  <= mem[addr];
                valid <= 1'b1;
            end
        end
    end

endmodule
```

Piso

piso.v

```
module piso_reg #(
    parameter WIDTH = 20,
    parameter ADDR_WIDTH = 8
)(
    input  wire                  clk,
    input  wire                  rst_n,
    input  wire [WIDTH-1:0]      parallel_in,
    input  wire                  ram_valid,
    output reg                   en,
    output reg  [ADDR_WIDTH-1:0] raddr,
    output reg                   serial_out,
    output reg                   valid
);

    reg [WIDTH-1:0] shift_reg;
    reg [ADDR_WIDTH-1:0] address;
    reg [5:0] count;
    reg state;

    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            shift_reg  <= {WIDTH{1'b0}};
            address    <= {ADDR_WIDTH{1'b0}};
            raddr      <= {ADDR_WIDTH{1'b0}};
            count      <= 6'd0;
            en         <= 1'b1;
            serial_out <= 1'b0;
            valid      <= 1'b0;
            state      <= 1'b0;
        end
        else begin
            if (state == 1'b0) begin
                en         <= 1'b0;
                serial_out <= 1'b0;
                valid      <= 1'b0;

                if (ram_valid) begin
                    shift_reg <= parallel_in;
                    state     <= 1'b1;
                end
            end
            else begin
                en         <= 1'b0;
                valid      <= 1'b1;
                serial_out <= shift_reg[WIDTH-1];

                shift_reg <= shift_reg << 1;

                if (count == 6'd1) begin
                    count <= 6'd0;
                    valid <= 1'b0;
                    serial_out <= 1'b0;
                    address <= address + 1'b1;
                    raddr <= address + 1'b1;
                    en <= 1'b1;
                    state <= 1'b0;
                end
                else begin
                    count <= count - 1'b1;
                end
            end
        end
    end

```

SIPO

sipo_reg.v

```v
module sipo_reg #(
    parameter WIDTH = 20
)(
    input  wire             clk,
    input  wire             rst_n,
    input  wire             shift_en,
    input  wire             serial_in,
    output reg [WIDTH-1:0]  parallel_out
);

    always @(posedge clk or negedge rst_n) begin
        if (!rst_n)
            parallel_out <= {WIDTH{1'b0}};
        else if (shift_en)
            parallel_out <= {parallel_out[WIDTH-2:0], serial_in};
    end

endmodule
```

ALU

alu.v

```v
module alu #(
    parameter WIDTH = 8
)(
    input  wire [WIDTH-1:0] in_a,
    input  wire [WIDTH-1:0] in_b,
    input  wire [2:0]       opcode,
    input  wire             alu_en,
    output reg  [WIDTH-1:0] alu_out,
    output wire             a_is_zero
);

    assign a_is_zero = (in_a == {WIDTH{1'b0}});

    always @(*) begin
        if (!alu_en) begin
            alu_out = {WIDTH{1'b0}};
        end
        else begin
            case (opcode)
                3'b000: alu_out = in_a + in_b;
                3'b001: alu_out = in_a - in_b;
                3'b010: alu_out = in_a & in_b;
                3'b011: alu_out = in_a ^ in_b;
                3'b100: alu_out = in_a | in_b;
                3'b101: alu_out = in_a;
                default: alu_out = {WIDTH{1'b0}};
            endcase
        end
    end

endmodule
```

Top

top ram piso sipo alu.v

```v
module top #(
    parameter ADDR_WIDTH = 8,
    parameter DATA_WIDTH = 20,
    parameter ALU_WIDTH = 8
)(
    input  wire                     clk,
    input  wire                     rst_n,
    input  wire                     wr_en,
    input  wire [ADDR_WIDTH-1:0]    wr_addr,
    input  wire [DATA_WIDTH-1:0]    din,
    output wire [ALU_WIDTH-1:0]     alu_out,
    output wire                     a_is_zero,
    output wire [DATA_WIDTH-1:0]    parallel_out,
    output wire                     serial_out,
    output wire                     valid
);

    wire [DATA_WIDTH-1:0] ram_dout;
    wire ram_valid;
    wire ram_rd_en;
    wire [ADDR_WIDTH-1:0] ram_raddr;

    wire alu_en;
    wire [2:0] opcode;
    wire [ALU_WIDTH-1:0] in_a;
    wire [ALU_WIDTH-1:0] in_b;

    ram #(
        .ADDR_WIDTH(ADDR_WIDTH),
        .DATA_WIDTH(DATA_WIDTH)
    ) u_ram (
        .clk(clk),
        .rst_n(rst_n),
        .wr_en(wr_en),
        .addr(wr_addr),
        .din(din),
        .rd_en(ram_rd_en),
        .dout(ram_dout),
        .valid(ram_valid)
```

```
    );

    piso_reg #(
        .WIDTH(DATA_WIDTH),
        .ADDR_WIDTH(ADDR_WIDTH)
    ) u_piso (
        .clk(clk),
        .rst_n(rst_n),
        .parallel_in(ram_dout),
        .ram_valid(ram_valid),
        .en(ram_rd_en),
        .raddr(ram_raddr),
        .serial_out(serial_out),
        .valid(valid)
    );

    sipo_reg #(
        .WIDTH(DATA_WIDTH)
    ) u_sipo (
        .clk(clk),
        .rst_n(rst_n),
        .shift_en(valid),
        .serial_in(serial_out),
        .parallel_out(parallel_out)
    );

    assign alu_en = parallel_out[19];
    assign opcode = parallel_out[18:16];
    assign in_a   = parallel_out[15:8];
    assign in_b   = parallel_out[7:0];

    alu #(
        .WIDTH(ALU_WIDTH)
    ) u_alu (
        .in_a(in_a),
        .in_b(in_b),
        .opcode(opcode),
        .alu_en(alu_en),
```

```
        .alu_out(alu_out),
        .a_is_zero(a_is_zero)
    );

endmodule
```

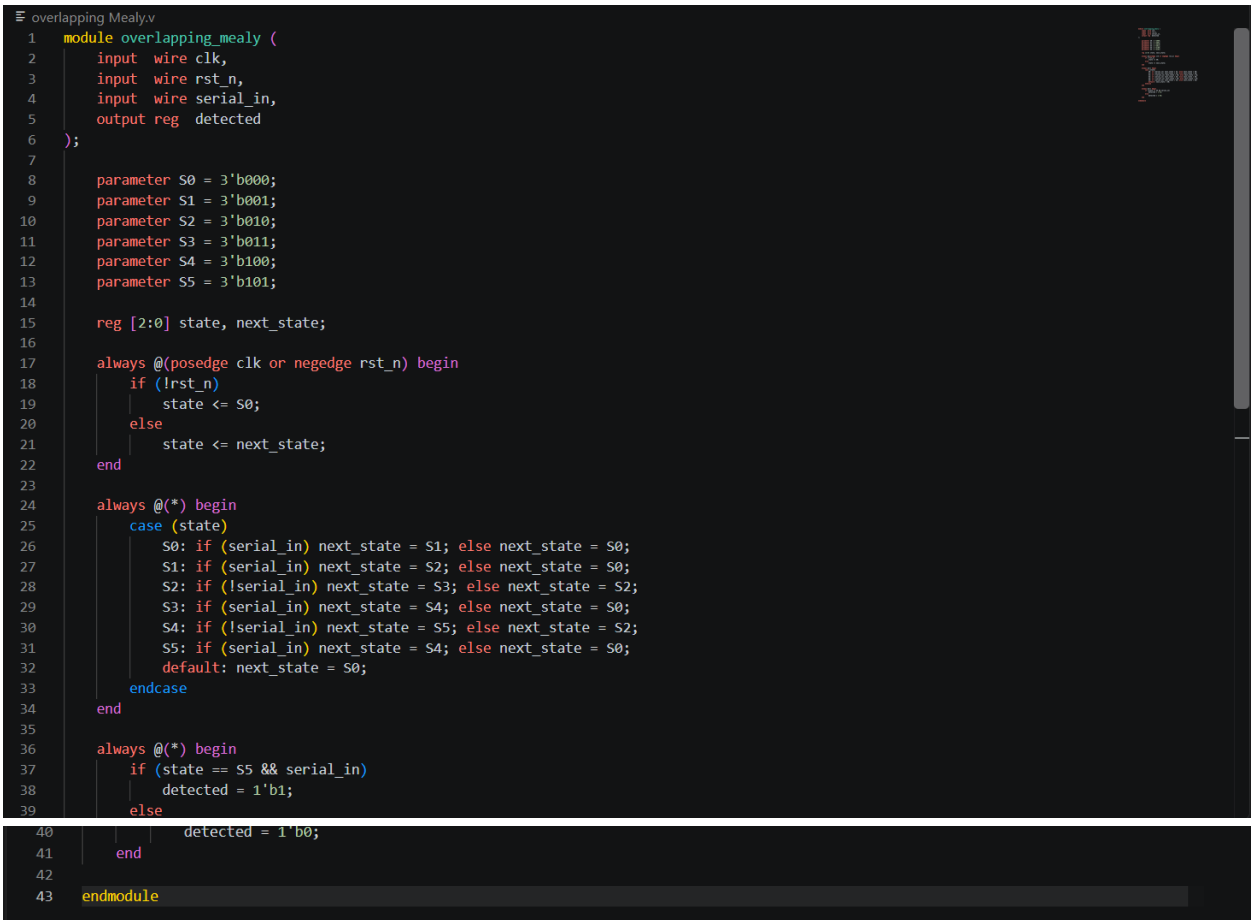
TB

tbLab3.v

```v
module tb_top;

    reg clk;
    reg rst_n;
    reg wr_en;
    reg [7:0] wr_addr;
    reg [19:0] din;

    wire [7:0] alu_out;
    wire a_is_zero;
    wire [19:0] parallel_out;
    wire serial_out;
    wire valid;

    top dut (
        .clk(clk),
        .rst_n(rst_n),
        .wr_en(wr_en),
        .wr_addr(wr_addr),
        .din(din),
        .alu_out(alu_out),
        .a_is_zero(a_is_zero),
        .parallel_out(parallel_out),
        .serial_out(serial_out),
        .valid(valid)
    );

    always #5 clk = ~clk;

    task write_ram;
        input [7:0] address;
        input [19:0] data;
        begin
            @(negedge clk);
            wr_en = 1'b1;
            wr_addr = address;
            din = data;
            @(negedge clk);
            wr_en = 1'b0;
        end
    endtask

    initial begin
        clk = 1'b0;
        rst_n = 1'b0;
        wr_en = 1'b0;
        wr_addr = 8'd0;
        din = 20'd0;

        #20;
        rst_n = 1'b1;

        write_ram(8'd0, 20'b1_000_00001010_00000101);
        write_ram(8'd1, 20'b1_001_00010100_00000101);
        write_ram(8'd2, 20'b1_010_00001111_00000101);
        write_ram(8'd3, 20'b1_011_00001111_00000101);
        write_ram(8'd4, 20'b1_100_00001111_00000101);
        write_ram(8'd5, 20'b1_101_00001111_00000101);
        write_ram(8'd6, 20'b0_000_00001111_00000101);
        write_ram(8'd7, 20'b1_110_00001111_00000101);

        #1000;

        $stop;
    end

endmodule
```

Last Lab with Eng. Mohamed (Note: I presented the rest of them to him in the session):

Overlapping sequence analyzer (mealy)



overlapping Mealy.v

```
module overlapping_mealy (
    input  wire clk,
    input  wire rst_n,
    input  wire serial_in,
    output reg  detected
);

    parameter S0 = 3'b000;
    parameter S1 = 3'b001;
    parameter S2 = 3'b010;
    parameter S3 = 3'b011;
    parameter S4 = 3'b100;
    parameter S5 = 3'b101;

    reg [2:0] state, next_state;

    always @(posedge clk or negedge rst_n) begin
        if (!rst_n)
            state <= S0;
        else
            state <= next_state;
    end

    always @(*) begin
        case (state)
            S0: if (serial_in) next_state = S1; else next_state = S0;
            S1: if (serial_in) next_state = S2; else next_state = S0;
            S2: if (!serial_in) next_state = S3; else next_state = S2;
            S3: if (serial_in) next_state = S4; else next_state = S0;
            S4: if (!serial_in) next_state = S5; else next_state = S2;
            S5: if (serial_in) next_state = S4; else next_state = S0;
            default: next_state = S0;
        endcase
    end

    always @(*) begin
        if (state == S5 && serial_in)
            detected = 1'b1;
        else
            detected = 1'b0;
    end

endmodule
```

Overlapping sequence analyzer (moore)

overlapping moore.v

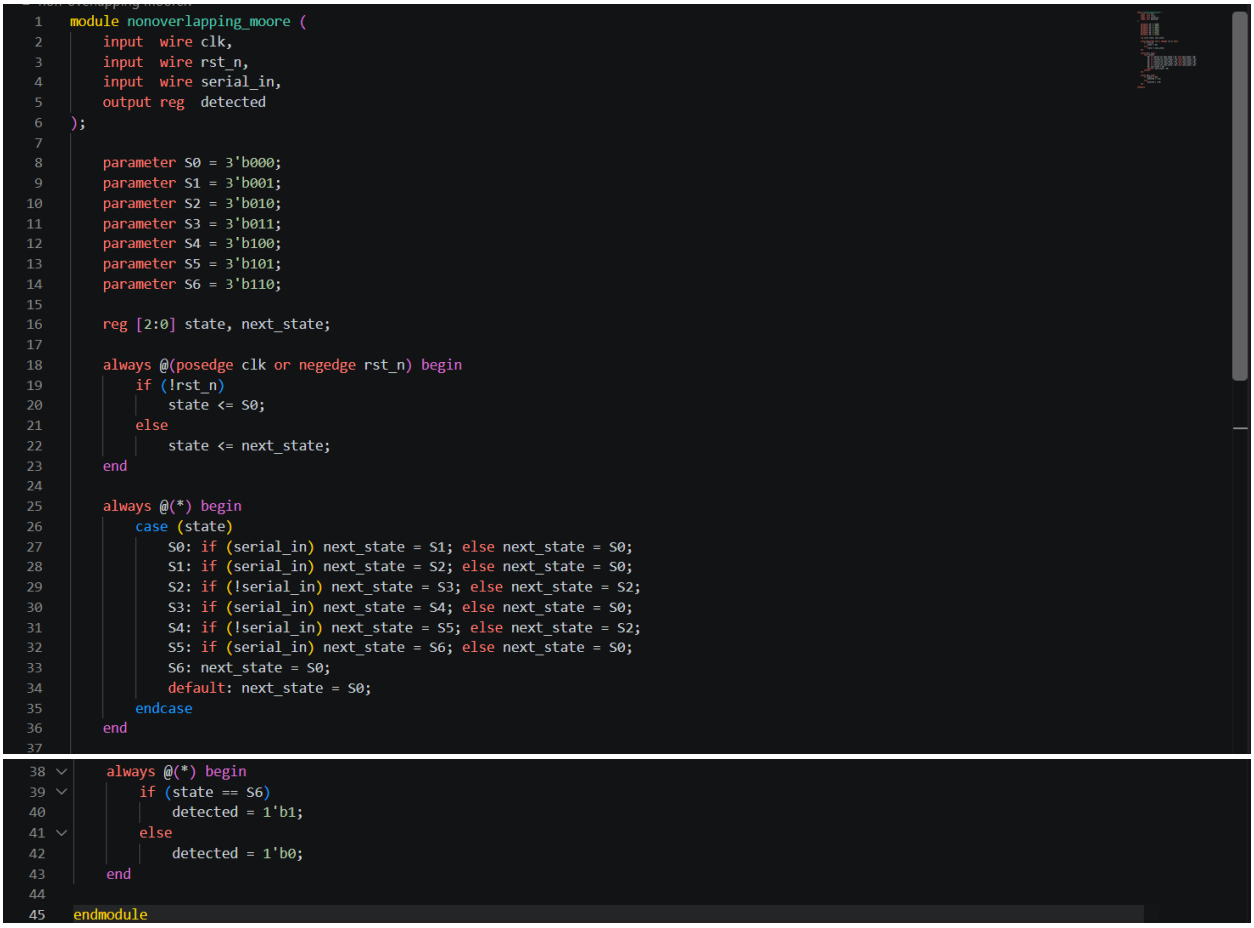
```v
module overlapping_moore (
    input  wire clk,
    input  wire rst_n,
    input  wire serial_in,
    output reg  detected
);

    parameter S0 = 3'b000;
    parameter S1 = 3'b001;
    parameter S2 = 3'b010;
    parameter S3 = 3'b011;
    parameter S4 = 3'b100;
    parameter S5 = 3'b101;
    parameter S6 = 3'b110;

    reg [2:0] state, next_state;

    always @(posedge clk or negedge rst_n) begin
        if (!rst_n)
            state <= S0;
        else
            state <= next_state;
    end

    always @(*) begin
        case (state)
            S0: if (serial_in) next_state = S1; else next_state = S0;
            S1: if (serial_in) next_state = S2; else next_state = S0;
            S2: if (!serial_in) next_state = S3; else next_state = S2;
            S3: if (serial_in) next_state = S4; else next_state = S0;
            S4: if (!serial_in) next_state = S5; else next_state = S2;
            S5: if (serial_in) next_state = S6; else next_state = S0;
            S6: if (serial_in) next_state = S4; else next_state = S0;
            default: next_state = S0;
        endcase
    end

    always @(*) begin
        if (state == S6)
            detected = 1'b1;
        else
            detected = 1'b0;
    end

endmodule
```

Non-overlapping sequence analyzer (mealy)

non-overlapping mealy.v

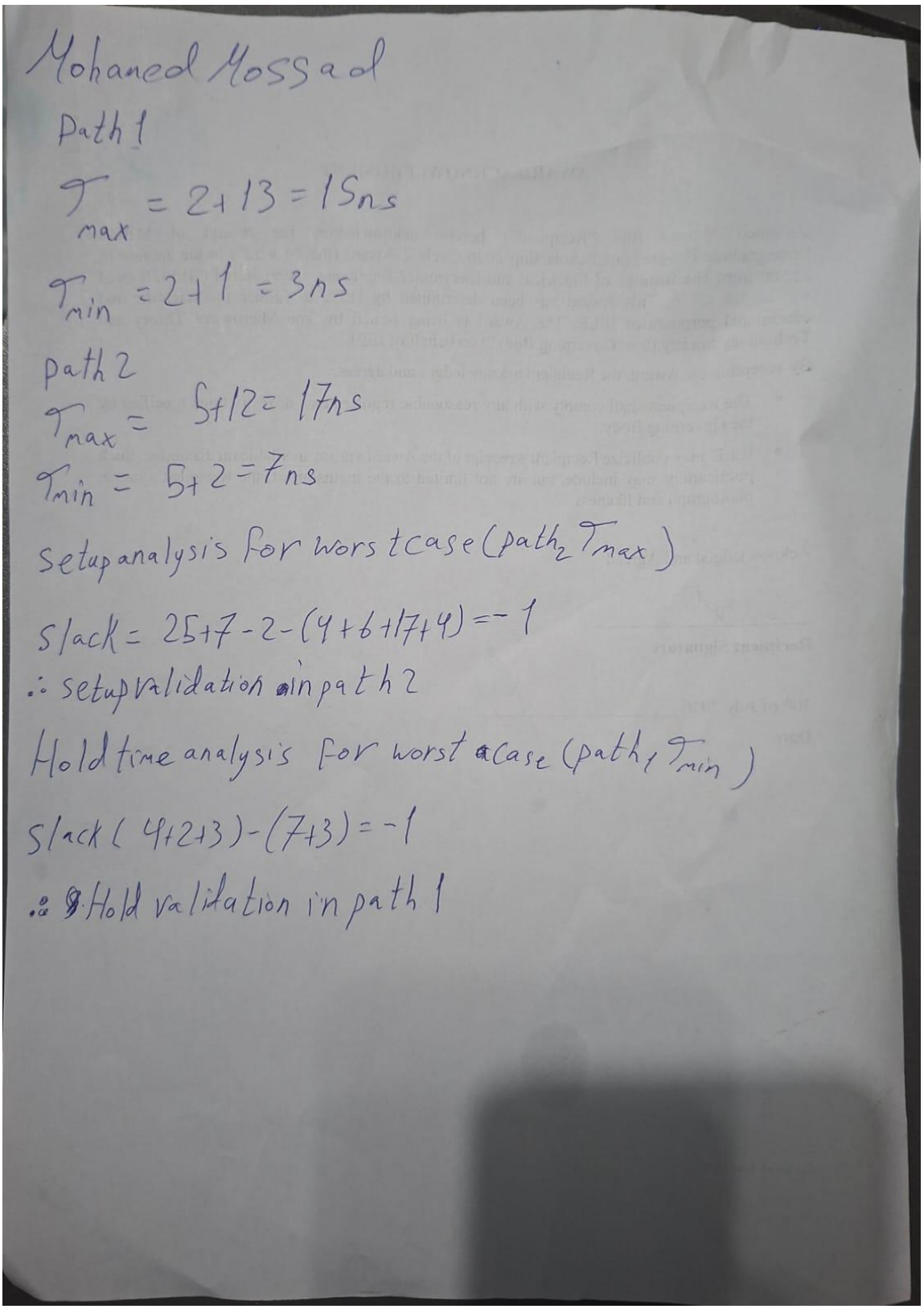
```v
module nonoverlapping_mealy (
    input  wire clk,
    input  wire rst_n,
    input  wire serial_in,
    output reg  detected
);

    parameter S0 = 3'b000;
    parameter S1 = 3'b001;
    parameter S2 = 3'b010;
    parameter S3 = 3'b011;
    parameter S4 = 3'b100;
    parameter S5 = 3'b101;

    reg [2:0] state, next_state;

    always @(posedge clk or negedge rst_n) begin
        if (!rst_n)
            state <= S0;
        else
            state <= next_state;
    end

    always @(*) begin
        case (state)
            S0: if (serial_in) next_state = S1; else next_state = S0;
            S1: if (serial_in) next_state = S2; else next_state = S0;
            S2: if (!serial_in) next_state = S3; else next_state = S2;
            S3: if (serial_in) next_state = S4; else next_state = S0;
            S4: if (!serial_in) next_state = S5; else next_state = S2;
            S5: if (serial_in) next_state = S0; else next_state = S0;
            default: next_state = S0;
        endcase
    end

    always @(*) begin
        if (state == S5 && serial_in)
            detected = 1'b1;
        else
            detected = 1'b0;
    end

endmodule
```

Non-overlapping sequence analyzer (moore)



```
module nonoverlapping_moore (
    input  wire clk,
    input  wire rst_n,
    input  wire serial_in,
    output reg  detected
);

    parameter S0 = 3'b000;
    parameter S1 = 3'b001;
    parameter S2 = 3'b010;
    parameter S3 = 3'b011;
    parameter S4 = 3'b100;
    parameter S5 = 3'b101;
    parameter S6 = 3'b110;

    reg [2:0] state, next_state;

    always @(posedge clk or negedge rst_n) begin
        if (!rst_n)
            state <= S0;
        else
            state <= next_state;
    end

    always @(*) begin
        case (state)
            S0: if (serial_in) next_state = S1; else next_state = S0;
            S1: if (serial_in) next_state = S2; else next_state = S0;
            S2: if (!serial_in) next_state = S3; else next_state = S2;
            S3: if (serial_in) next_state = S4; else next_state = S0;
            S4: if (!serial_in) next_state = S5; else next_state = S2;
            S5: if (serial_in) next_state = S6; else next_state = S0;
            S6: next_state = S0;
            default: next_state = S0;
        endcase
    end

    always @(*) begin
        if (state == S6)
            detected = 1'b1;
        else
            detected = 1'b0;
    end

endmodule
```

STA Task:



Mohaned Mossad

Path 1

$\tau_{max} = 2+13 = 15ns$

$\tau_{min} = 2+1 = 3ns$

path 2

$\tau_{max} = 5+12 = 17ns$

$\tau_{min} = 5+2 = 7ns$

setup analysis for worst case ($path_2$ τ_{max})

slack = 25+7-2-(4+6+17+4) = -1

∴ setup validation in path 2

Hold time analysis for worst case ($path_1$ τ_{min})

slack (4+2+3)-(7+3) = -1

∴ Hold validation in path 1

System Verilog Lab:

Q1)

```
module array_partition;
  int arr[] = '{9, 7, 4, 6, 2, 8, 6, 5};
  int even_arr[];
  int odd_arr[];

  initial begin
    even_arr = arr.find(x) with (x % 2 == 0);
    odd_arr  = arr.find(x) with (x % 2 != 0);
    $display("Even Array:     %p", even_arr);
    $display("Odd Array:      %p", odd_arr);
  end
endmodule
```

Q2)

system verilog > maximum consecuative number.sv

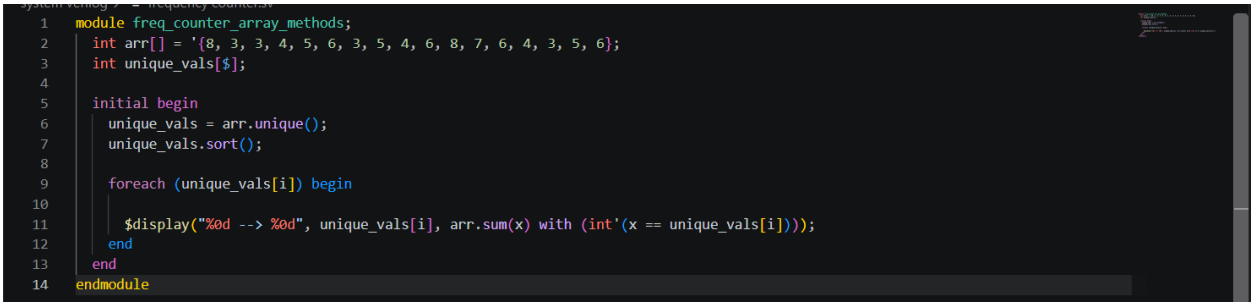
```
module freq_zeros_vs_ones;
  int arr[] = '{1, 1, 0, 1, 1, 1, 1, 0, 0, 0, 1};
  int ones_count;
  int zeros_count;
  int c;

  initial begin
    foreach (arr[i]) begin
      c = (i == 0 || arr[i] != arr[i-1]) ? 1 : c + 1;

      if (arr[i] == 1 && c > ones_count) ones_count = c;
      if (arr[i] == 0 && c > zeros_count) zeros_count = c;
    end

    if (ones_count > zeros_count)
      $display("1 --> %0d", ones_count);
    else if (zeros_count > ones_count)
      $display("0 --> %0d", zeros_count);
    else
      $display("0s and 1s are equal");
  end
endmodule
```

Q3)

system verilog > second maximum number.sv

```
module second_max_array_method;
  int arr[] = '{45, 34, 67, 89, 78};

  int max_val[$];
  int filtered_q[$];
  int sec_max[$];

  initial begin
    max_val = arr.max();
    filtered_q = arr.find(x) with (x < max_val[0]);
    sec_max = filtered_q.max();
    $display("Second Max Number: %0d", sec_max[0]);
  end
endmodule
```

Q4)



```
module freq_counter_array_methods;
  int arr[] = '{8, 3, 3, 4, 5, 6, 3, 5, 4, 6, 8, 7, 6, 4, 3, 5, 6};
  int unique_vals[$];

  initial begin
    unique_vals = arr.unique();
    unique_vals.sort();

    foreach (unique_vals[i]) begin

      $display("%0d --> %0d", unique_vals[i], arr.sum(x) with (int'(x == unique_vals[i])));
    end
  end
endmodule
```

Thank you Eng. Omar for your effort I really benifited a lot.